



Portero Seguro

Sistema de Control de Activos y Trazabilidad

Documentación técnica extendida: arquitectura, ciclo de vida de la petición, modelo de datos a nivel de columna, seguridad, casos de uso con validaciones reales, lógica de negocio, capa de presentación, generación de PDF y despliegue en red. Sistema de inventario corporativo para CCTV, control de accesos, redes y consumibles, con trazabilidad por número de serie, guías de salida, kardex auditable y operación en campo.

Python + Flask

SQLite · 18 tablas

Caddy · proxy inverso

~50 rutas

19 pantallas

3 roles

ReportLab · PDF

Tabla de contenidos

1	Introducción y alcance	3
2	Glosario del dominio	3
3	Arquitectura general	4
4	Ciclo de vida de una petición	5
5	Stack tecnológico	6
6	Seguridad y control de acceso	7
7	Roles y matriz de permisos	9
8	Modelo de datos — visión y diagrama ER	10
9	Diccionario de datos (columna a columna)	11
10	Módulos y casos de uso detallados	14
11	Lógica de negocio clave	18
12	Tipos de movimiento (kardex)	20
13	Capa de presentación (frontend y PDF)	21
14	Mapa de rutas (endpoints)	22
15	Despliegue y acceso en red	23
16	Estructura, pruebas, roadmap y apéndices	25

18
TABLAS

~50
RUTAS

19
PANTALLAS

8
TIPOS DE MOV.

10
TESTS

3
ROLES

1 · Introducción y alcance

Portero Seguro es una aplicación web de **gestión de inventario y trazabilidad de activos** para una empresa de instalaciones de seguridad electrónica (CCTV, control de accesos, redes y consumibles). Centraliza el almacén, el despacho a obra mediante guías de salida, el seguimiento de equipos en campo y la bitácora de actividades, con control de stock y auditoría de cada movimiento.

Alcance funcional

- Catálogo de productos por **cantidad** y por **número de serie**.
- Ingresos (unitarios, masivos serializados y por cantidad con unificación).
- Guías de salida corporativas con PDF, edición, anulación y devolución parcial.
- Salidas directas simples.
- Kardex de movimientos (auditoría completa con usuario y fecha).
- Seguimiento de equipos y herramientas en campo.
- Bitácora de avances / actividades de obra.
- Administración: catálogos, edificios, personal, usuarios y roles.

Fuera de alcance (por diseño actual)

No incluye facturación, compras/órdenes de compra, integración contable ni multi-almacén. El despliegue objetivo es **una sola máquina servidor** en red local, con SQLite como almacén embebido (adecuado para un único proceso y baja-media concurrencia).

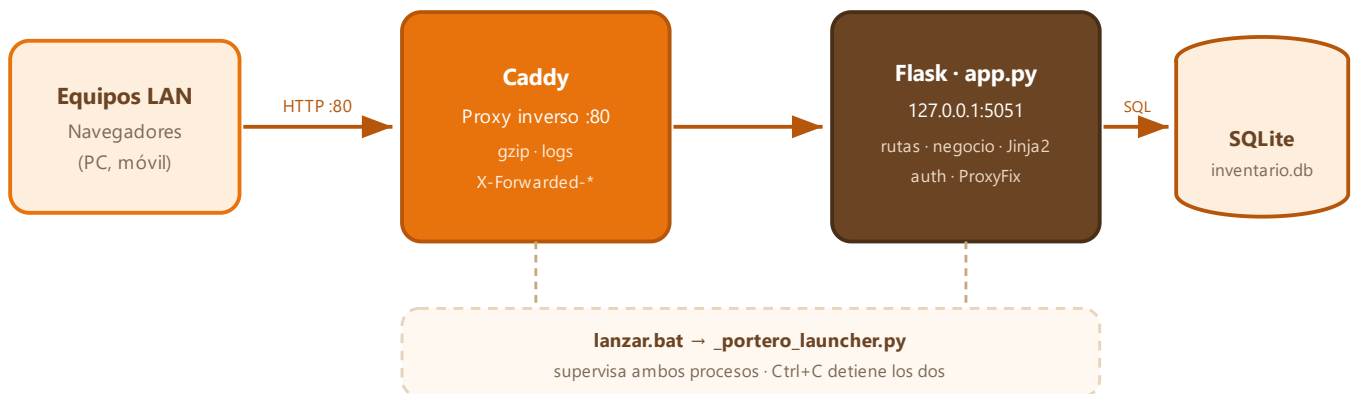
2 · Glosario del dominio

Término	Significado en el sistema
Producto base	Fila de equipos . Se identifica por Categoría + Marca + Modelo(descripción) + tipo de control. Agrupa el stock.
control_stock	Modo del producto: CANTIDAD (consumibles) o SERIAL (unidades rastreadas por serie).
Serie / unidad	Registro en equipo_series : una unidad física con serial único, estado y ubicación actual.
Guía de salida	Documento de despacho (cabecera + líneas + series) hacia un edificio/cliente. Genera movimientos y PDF.
Kardex / movimiento	Registro inmutable de cada cambio de stock en movimientos (tipo, cantidad, stock antes/después, usuario).
Seguimiento	Equipo/herramienta dejado temporalmente en un edificio o cliente, pendiente de retiro/devolución.
Avance / nota	Actividad de obra registrada en bitácora, con estado y responsable.
Destino / edificio	Sede u obra (edificios) hacia la que se despacha o donde se instala.

3 · Arquitectura general

Aplicación monolítica Flask, servida tras un proxy inverso, con base de datos embebida.

Flask atiende solo en `127.0.0.1:5051` (nunca directo a la red) y **Caddy** es la única cara pública en el puerto 80, reenviando el tráfico. Todo corre en una máquina servidor con SQLite.



Descomposición en módulos de código

Archivo	Responsabilidad	Contenido
<code>app.py</code>	Controlador / negocio	~3.450 líneas: rutas, validaciones, orquestación de transacciones, helpers de stock y guías.
<code>db.py</code>	Capa de datos	Conexión con PRAGMAs, esquema, migraciones idempotentes, helpers de catálogos.
<code>auth.py</code>	Seguridad	Hash de contraseñas, jerarquía de roles, tokens CSRF. Sin acoplarse a Flask.
<code>pdf_render.py</code>	Presentación PDF	Maquetación ReportLab de la guía de salida; devuelve un <code>BytesIO</code> .
<code>migrar_fase1.py</code>	Datos históricos	Consolidación de productos duplicados y reapuntado de referencias.

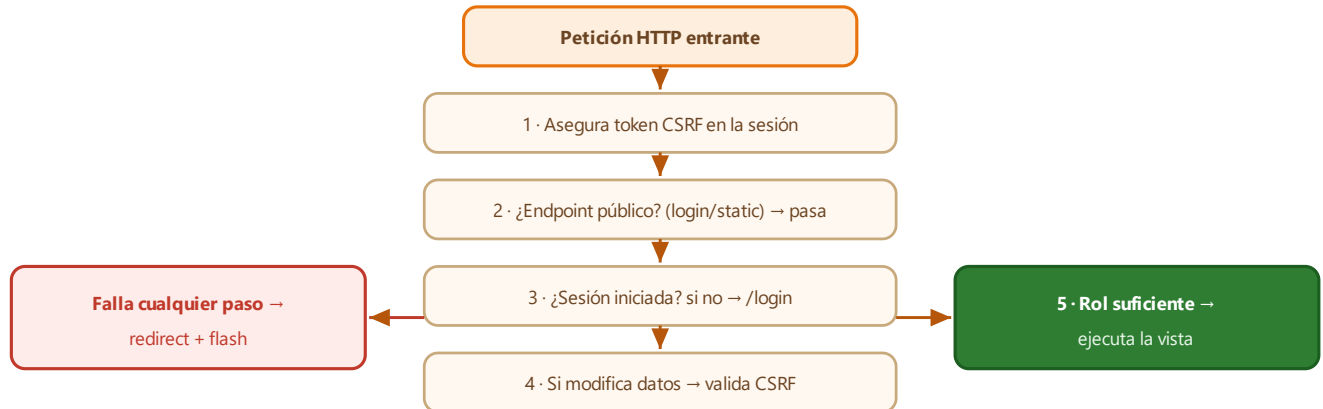
Principios de diseño

- **Monolito pragmático:** una sola app Flask; la complejidad transversal (datos, seguridad, PDF) está desacoplada en módulos.
- **SQL directo, sin ORM:** consultas explícitas; el esquema se crea/migra en cada arranque (`init_db`) de forma idempotente.
- **Render en servidor:** Jinja2 (19 vistas) + JavaScript vanilla; sin framework frontend.
- **Flask aislado:** el proxy inverso es la superficie de red; la app nunca se expone directa.
- **Transaccional:** las operaciones críticas (guías, bajas) abren transacción y hacen `rollback` ante cualquier error.

4 · Ciclo de vida de una petición

Toda petición pasa por un único guardián de seguridad antes de llegar a la vista.

El decorador `@app.before_request` (`seguridad_global`) centraliza autenticación, CSRF y autorización. Este es el orden exacto de comprobaciones:



Paso	Qué hace	Si falla
CSRF por sesión	Crea <code>session['csrf_token']</code> si no existe (protege incluso el login).	—
Endpoints públicos	<code>login</code> y <code>static</code> se dejan pasar sin sesión.	—
Autenticación	Exige <code>session['user_id']</code> . En GET guarda <code>?next=</code> para volver.	Redirige a <code>/login</code>
Validación CSRF	En <code>POST/PUT/PATCH/DELETE</code> compara el token con <code>secrets.compare_digest</code> .	Redirige con aviso
Autorización	Rol mínimo de <code>ROUTE_ACCESS</code> (por método); no listado ⇒ <code>admin</code> .	Redirige a <code>/</code>

Un `@app.context_processor` inyecta en todas las plantillas `csrf_token` , `usuario_actual` y `rol_actual` , de modo que la UI puede adaptar menús y el JS puede insertar el token en los formularios (ver §13).

Deny-by-default. La autorización parte de "denegar salvo que se liste explícitamente": si se agrega una ruta nueva y se olvida registrarla en `ROUTE_ACCESS` , queda automáticamente restringida a `admin` en lugar de quedar abierta.

5 · Stack tecnológico

Capa	Tecnología	Rol y notas
Lenguaje	Python 3.10+	Runtime del backend.
Framework	Flask ≥ 3.0	Enrutado, sesiones firmadas, plantillas, <code>flash</code> .
Plantillas	Jinja2	19 vistas server-side con herencia de <code>base.html</code> .
Seguridad	Werkzeug	<code>generate/check_password_hash</code> (PBKDF2), <code>ProxyFix</code> .
Base de datos	SQLite 3	Archivo <code>inventario.db</code> ; PRAGMAs <code>foreign_keys</code> y <code>busy_timeout=5000</code> .
PDF	ReportLab ≥ 4.0	Guía de salida imprimible (Platypus + flowables propios).
Proxy inverso	Caddy	Binario único; HTTP :80 → Flask. Se descarga en el primer arranque.
Frontend	JS vanilla + CSS	<code>app.js</code> , <code>dashboard.js</code> ; sin dependencias de build.
Arranque	Batch + launcher Python	<code>lanzar.bat</code> , <code>_portero_launcher.py</code> .
Pruebas	unittest	Integración vía cliente de test de Flask y BD temporal.

Dependencias Python declaradas en `requirements.txt`: `Flask≥3.0.0` y `reportlab≥4.0.0` (Werkzeug y Jinja2 vienen con Flask). Sin ORM, sin Node/npm, sin servicios externos.

Decisiones y sus razones

Decisión	Motivo
SQLite (no cliente/servidor)	Despliegue de una máquina, cero administración de BD, respaldo = copiar un archivo.
SQL directo (no ORM)	Consultas de inventario/guías con joins específicos; menos capas, control total del rendimiento.
Migraciones idempotentes en arranque	Instalación y actualización sin herramienta aparte; se añaden columnas/índices si faltan.
Proxy inverso Caddy	URL limpia por dominio, Flask aislado en localhost, base lista para HTTPS futuro.

6 · Seguridad y control de acceso

De "cualquiera con la URL lo modificaba todo" a autenticación, roles, CSRF y auditoría.

El proyecto original no tenía autenticación, corría con el debugger expuesto y borraba datos por `GET`. La versión actual incorpora una capa de seguridad completa:

#	Control	Cómo funciona
1	Login obligatorio	Solo <code>/login</code> y <code>static</code> son públicos; el resto exige sesión.
2	3 roles jerárquicos	<code>lectura(0) < operador(1) < admin(2)</code> ; se compara por nivel numérico.
3	Hash PBKDF2	Werkzeug con salt por contraseña; nunca texto plano. Cambio propio exige ≥ 8 caracteres.
4	CSRF	Token por sesión; validado con <code>compare_digest</code> en todo método que escribe.
5	Rate limiting login	Clave <code>IP:usuario</code> , 5 fallos / 300 s en memoria; luego rechaza aun con clave correcta.
6	Deny-by-default	Endpoint no listado en <code>ROUTE_ACCESS</code> $\Rightarrow$ exige <code>admin</code> .
7	Escrituras por POST	Anular guía y eliminar edificio dejaron de ser <code>GET</code> (evita CSRF por enlace).
8	Auditoría real	Cada movimiento guarda el usuario autenticado de la sesión, no un valor fijo.
9	SECRET_KEY sin default	De entorno o generada una vez en <code>.secret_key</code> (fuera de git).
10	Debug apagado	Arranque seguro; debug/host abierto solo por variable de entorno.
11	Descuento atómico	<code>UPDATE ... WHERE cantidad $\geq$ n</code> : imposible dejar stock negativo (ver §11).
12	Logging rotativo	Logins (ok/fallo/bloqueo con IP), guías, bajas y excepciones en <code>logs/app.log</code> .
13	Cookies endurecidas	<code>HttpOnly</code> , <code>SameSite=Lax</code> , <code>Secure</code> configurable.

Flujo de inicio de sesión

1. Comprueba rate limit por `IP:usuario`; si está bloqueado, avisa y corta.
2. Busca el usuario con `COLLATE NOCASE` (login insensible a mayúsculas).
3. Verifica que exista, esté `activo` y que el hash de contraseña coincida.
4. En caso de fallo: registra el intento (rate limit) y lo escribe en el log; mensaje genérico "usuario o contraseña incorrectos".
5. En caso de éxito: limpia rate limit, `session.clear()`, guarda `user_id/username/nombre/rol/csrf_token`, marca la sesión permanente y actualiza `ultimo_acceso`.
6. Si `debe_cambiar_password`, redirige a Usuarios para forzar el cambio; si hay `?next=` interno, vuelve allí.

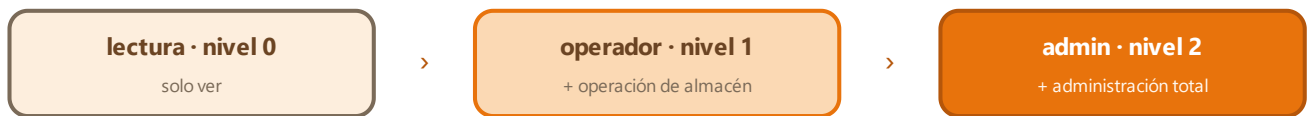
Gestión de contraseñas

- **Primer arranque:** con `usuarios` vacía se crea un `admin`; sin variables de entorno, la contraseña es aleatoria y se imprime **una sola vez** en consola.
- **Cambio propio:** exige la contraseña actual, mínimo 8 caracteres y confirmación coincidente; limpia `debe_cambiar_password`.
- **Alta por admin:** se genera una contraseña temporal legible; el usuario debe cambiarla en su primer ingreso.
- **Reseteo por admin:** vuelve a generar temporal y re-activa `debe_cambiar_password`.

Nota de despliegue actual (HTTP). En el modo de red local sin cifrado, `SESSION_COOKIE_SECURE` queda en `false` a propósito (si fuera `true`, el navegador no enviaría la cookie sobre HTTP y el login fallaría). Ver §15 y el roadmap para HTTPS.

7 · Roles y matriz de permisos

cada rol hereda todo lo del anterior



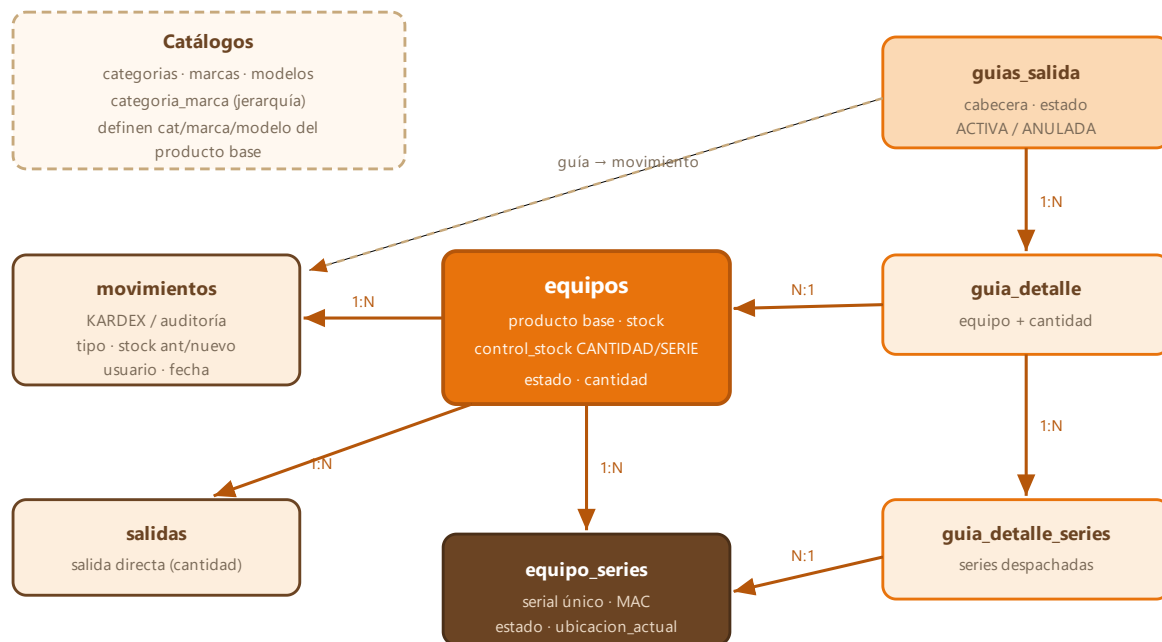
Acción	lectura	operador	admin
Ver dashboard, guías, movimientos, series	✓	✓	✓
Registrar ingresos / salidas / guías	✗	✓	✓
Anular guía · devolver series · dar de baja	✗	✓	✓
Seguimiento y avances de obra	✗	✓	✓
Catálogos (categorías, marcas, modelos, cargos)	✗	✗	✓
Personal y edificios (crear/editar/eliminar)	✗	✗	✓
Gestión de usuarios del sistema	✗	✗	✓

La autorización se resuelve en un único punto contra `ROUTE_ACCESS`, que admite exigir un rol distinto por método HTTP: una pantalla se *ve* con `lectura` pero se *modifica* con `operador` o `admin`.

```
# Ejemplos reales del mapa (app.py)
'ingresos':      {'GET': 'lectura', 'POST': 'operador'},
'personal':      {'GET': 'lectura', 'POST': 'admin'},
'edificios':      {'GET': 'lectura', 'POST': 'admin'},
'configuracion': 'admin',
'guardar_guia':  'operador',
'dar_baja_equipo': 'operador',
# endpoint no listado → 'admin' (deny-by-default)
```


8 · Modelo de datos — visión y diagrama ER

SQLite con 18 tablas. Núcleo transaccional: productos, series, guías y movimientos.



Relaciones adicionales (fuera del dibujo): `seguimiento_equipos` → `equipos` / `equipo_series`; `personal` → `cargos`; `usuarios`, `edificios`, `seguimiento_herramientas` y `avances_actividades` son independientes.

Integridad y migraciones

- Cada conexión activa `PRAGMA foreign_keys = ON` y `busy_timeout = 5000` (tolera bloqueos transitorios).
- `init_db` crea las tablas si faltan y añade columnas/índices ausentes (`add_column_if_missing`) sin destruir datos: la actualización es solo desplegar el código y arrancar.
- Normalizaciones automáticas en el arranque: estados con tildes → sin tildes, estado por stock, y ubicación de series históricas.
- Siembra inicial: categorías/marcas base, cargos y personal de ejemplo, y el usuario `admin`.

9 · Diccionario de datos

Detalle columna a columna de las entidades principales (SQLite; PK = clave primaria autoincremental).

equipos — producto base del inventario

Columna	Tipo	Restricción / defecto	Descripción
id	INTEGER	PK	Identificador del producto.
categoria	TEXT	NOT NULL	Categoría (CCTV, Redes...).
marca	TEXT	NOT NULL	Marca (Hikvision, Cisco...).
descripcion	TEXT	NOT NULL	Modelo / nombre del producto.
sku	TEXT	—	SKU/serie cuando es unidad única.
mac	TEXT	—	Dirección MAC (validada por formato).
estado	TEXT	def 'En Stock'	Estado operativo (ver apéndice).
cantidad	INTEGER	NOT NULL def 0	Stock actual.
stock_minimo	INTEGER	NOT NULL def 0	Umbral de reposición.
control_stock	TEXT	NOT NULL def 'CANTIDAD'	CANTIDAD o SERIAL .
observaciones	TEXT	—	Notas libres.
fecha_creacion	TIMESTAMP	def now	Alta del producto.
fecha_actualizacion	TIMESTAMP	—	Último cambio.

equipo_series — unidad física serializada

Columna	Tipo	Restricción	Descripción
id	INTEGER	PK	Identificador de la unidad.
equipo_id	INTEGER	FK→equipos	Producto base al que pertenece.
serial	TEXT	UNIQUE NOT NULL	Número de serie (único global).
mac	TEXT	—	MAC de la unidad.
estado	TEXT	def 'EN_STOCK'	EN_STOCK / ENTREGADO / BAJA .
guia_id	INTEGER	FK→guias_salida	Guía que la despachó (si aplica).
ubicacion_actual	TEXT	—	Almacén, destino, "Dado de baja"...
fecha_ingreso	TIMESTAMP	def now	Alta de la unidad.
observaciones	TEXT	—	Notas (se anexa el motivo de baja).

guias_salida — cabecera de despacho

Columna	Tipo	Restricción	Descripción
id	INTEGER	PK	Nº interno (código legible GS-000001).
personal	TEXT	NOT NULL	Solicitante (debe existir en personal).
destino	TEXT	NOT NULL	Edificio destino (debe existir).
cargo, proyecto	TEXT	—	Contexto del despacho.
entregado_por	TEXT	—	Quién entrega (almacén).

recibido_por	TEXT	—	Quién recibe.
aprobado_por	TEXT	—	Quién aprueba.
estado	TEXT	def 'ACTIVA'	ACTIVA / ANULADA .
fecha	TIMESTAMP	def now	Emisión.
fecha_anulacion, motivo_anulacion	TS/TEXT	—	Datos de anulación.

guia_detalle

id	PK
guia_id	FK→guias_salida
equipo_id	FK→equipos
cantidad	NOT NULL

guia_detalle_series

id	PK
guia_detalle_id	FK
serie_id	FK→equipo_series
UNIQUE(guia_detalle_id, serie_id)	

movimientos (kardex)

id	PK
equipo_id	FK→equipos
guia_id	FK→guias_salida
tipo	INGRESO/SALIDA_GUIA...
cantidad	NOT NULL
stock_anterior	NOT NULL
stock_nuevo	NOT NULL
referencia	código (GS-, BAJA-...)
usuario	def 'Sistema' → real
fecha	def now

usuarios — cuentas y roles

Columna	Tipo	Restricción	Descripción
id	INTEGER	PK	—
username	TEXT	UNIQUE NOT NULL	Login (insensible a mayúsculas).
password_hash	TEXT	NOT NULL	Hash PBKDF2.
nombre_completo	TEXT	NOT NULL	Nombre para mostrar.
rol	TEXT	def 'lectura'	lectura/operador/admin .
activo	INTEGER	def 1	0 = deshabilitado (no puede entrar).
debe_cambiar_password	INTEGER	def 0	Fuerza cambio en el próximo ingreso.
fecha_creacion	TIMESTAMP	def now	—
ultimo_acceso	TIMESTAMP	—	Se actualiza en cada login.

Tablas de operación en campo

seguimiento_equipos

equipo_id / serie_id	FK (opcionales)
nombre_equipo	NOT NULL
categoria/marca/modelo/serial	copia descriptiva
edificio	NOT NULL
ubicacion_detalle	—
fecha_dejado / dejado_por	NOT NULL
solicitado_por / motivo	—
estado	def EN_SEGUIMIENTO
fecha_retiro / retirado_por	—

avances_actividades

fecha	NOT NULL
actividad	NOT NULL
personal	NOT NULL
solicitado_por	—
edificio / proyecto	—
estado	def EN_PROCESO
detalles / observaciones	—

seguimiento_herramientas

herramienta / personal	NOT NULL
edificio / fecha_dejado	NOT NULL
estado / fecha_retorno	seguimiento

Catálogos y maestros

Tabla	Claves	Uso
categorias	nombre UNIQUE	Categorías de producto.
marcas	nombre UNIQUE	Marcas.
modelos	UNIQUE(nombre,categoria,marca)	Modelos válidos por cat+marca.
categoria_marca	UNIQUE(categoria,marca)	Jerarquía válida cat→marca.
cargos	nombre UNIQUE	Cargos del personal.
personal	nombre UNIQUE, cargo	Personal (solicita/recibe).
edificios	nombre UNIQUE	Sedes/obras; ubicacion, mapa_url.
salidas	equipo_id FK	Salida directa simple (personal, destino, cantidad).

Índices

Se indexan las búsquedas frecuentes: productos por `descripcion` , `categoria+marca+descripcion` y `sku` ; guías por `estado` ; detalle por guía; movimientos por `equipo` y `guia` ; series por `equipo+estado` y por `serial` ; seguimiento por `estado/edificio/equipo` ; avances por `fecha/estado/personal` ; y usuarios por `username` .

10 · Módulos y casos de uso detallados

Cada caso indica actor, precondiciones, campos, validaciones y resultado.

CU-01 · Ingresar producto por cantidad operador

Pantalla: `/ingresos` · **Precondición:** categoría, marca y modelo existentes y relacionados.

Campos: categoría, marca, descripción(modelo), sku, mac, cantidad, stock_mínimo, estado, observaciones.

Validaciones: la categoría y la marca existen; la marca está asociada a la categoría; el modelo pertenece a esa cat+marca; cantidad > 0; stock_mínimo ≥ 0; estado válido; MAC con formato; **si se informa SKU/serie o MAC, la cantidad debe ser 1** (para varios usar CU-02).

Resultado: si el producto base ya existe (misma cat+marca+modelo), **suma stock** sin duplicar; registra movimiento

INGRESO. Si trae SKU/MAC con cantidad 1, crea la unidad en `equipo_series` sin duplicar el producto base.

CU-02 · Ingreso masivo serializado operador

Pantalla: `/ingreso_series`.

Flujo: selecciona categoría → marca (filtrada) → modelo (filtrado) → cantidad esperada → pega los seriales (uno por línea; opcional `SERIAL,MAC`) → guardar.

Resultado: crea/actualiza el producto base (control_stock = SERIAL) y registra cada serial como unidad física en `equipo_series` con estado **EN_STOCK**. Los seriales duplicados se rechazan (serial único).

CU-03 · Crear guía de salida operador

Pantalla: `/guias` → `/guardar_guia`.

Campos cabecera: personal(solicitante), destino(edificio), cargo, proyecto, entregado/recibido/aprobado_por, observaciones. **Líneas:** productos (JSON) con cantidad; para productos SERIAL, las series exactas.

Validaciones: el solicitante y el destino existen; el JSON de productos es válido; cada producto y su stock/series se validan antes de escribir.

Transacción: inserta la guía (ACTIVA) y por cada línea: crea el detalle; si es SERIAL marca cada serie **ENTREGADO** y la enlaza; **descuenta stock atómicamente**; registra **SALIDA_GUIA**. Cualquier fallo hace **rollback** completo.

Resultado: guía con código `GS-000001`, stock descontado, series entregadas y PDF disponible.

CU-04 · Anular guía / devolver series operador

Rutas: `/eliminar_guia/<id>` (POST), `/guia/<id>/quitar_serie/<sid>`.

Anulación: estado → ANULADA; **reintegra el stock** de cada línea y devuelve las series a **EN_STOCK**; registra movimientos de reverso.

Devolución parcial: quitar una serie la libera a EN_STOCK, suma 1 al stock, registra **DEVOLUCION_GUIA** y elimina las líneas que quedan en cantidad 0.

CU-05 · Salida directa operador

Pantalla: `/salidas`. Descuento simple por cantidad (con el mismo descuento atómico). **Los productos serializados no salen por aquí:** deben ir por guía para elegir los seriales exactos.

CU-06 · Dar de baja producto operador

Ruta: `/dar_baja_equipo/<id>` (POST, con motivo). **No borra:** pone estado *Baja*, cantidad 0 y registra **BAJA** conservando historial.

Bloqueos: si el producto tiene **guías activas** → no se permite; si es SERIAL y hay series entregadas/instaladas → se bloquea hasta regularizarlas (las que estén EN_STOCK pasan a BAJA).

CU-07 · Seguimiento en campo operador

Pantalla: `/seguimiento`. Registra equipos dejados temporalmente en edificios/clientes (módems, UPS, cámaras...), con edificio, ubicación, quién lo dejó, motivo y estado (EN_SEGUIMIENTO / RETIRADO / INSTALADO / DEVUELTO / BAJA). Actualización y eliminación por rutas propias.

CU-08 · Avances / bitácora operador

Pantalla: `/avances`. Registra actividades por personal, edificio y proyecto, con estado (PENDIENTE / EN_PROCESO / TERMINADO / OBSERVADO / CANCELADO), detalles y observaciones.

CU-09 · Gestión de usuarios admin · autoservicio todos

Pantalla: `/usuarios`. El admin crea usuarios (rol + contraseña temporal), edita, resetea contraseñas y activa/desactiva. **Cualquier usuario** puede cambiar su propia contraseña (exige actual, mínimo 8, confirmación).

CU-10 · Catálogos y jerarquía admin

Pantalla: `/configuracion`. Administra categorías, relaciones categoría→marca y modelos por cat+marca; permite limpiar relaciones vacías. Los formularios de ingresos y guías consultan estas relaciones por API para filtrar en cascada (categoría → marca → modelo → producto).

CU-11 · Edificios y personal admin

Pantallas: `/edificios` (nombre, ubicación, link de mapa, observaciones) y `/personal` (nombre + cargo). Son los maestros que alimentan destinos y solicitantes de las guías y del seguimiento.

CU-12 · Consulta y trazabilidad lectura

Pantallas: Dashboard (/) con búsqueda instantánea y paginación; `/series` (estado y ubicación de cada unidad); `/movimientos` (kardex); `/listar_guias` y ver guía + PDF. Todo accesible en modo solo lectura.

11 · Lógica de negocio clave

11.1 · Descuento atómico de stock

En vez de *leer* → *validar* → *escribir* (inseguro bajo concurrencia), se usa un **UPDATE** condicionado que solo descuenta si el stock alcanza en el momento de la escritura:

```
UPDATE equipos
  SET cantidad = cantidad - :n
 WHERE id = :id AND cantidad >= :n;  -- éxito solo si rowcount == 1
```

Si dos despachos compiten por las últimas unidades, uno actualiza y el otro obtiene **rowcount = 0**: la guía se cancela con **rollback** y un mensaje claro, sin dejar stock negativo. Se aplica en guías y salidas.

11.2 · Ciclo de vida de una guía



La anulación revierte todo: stock reintegrado y cada serie de vuelta a disponible.

11.3 · Máquina de estados de una serie



11.4 · Producto unificado y tipos de control

- El producto base se identifica por **Categoría + Marca + Modelo + tipo de control**; ingresar el mismo producto **suma stock** en lugar de duplicar.
- CANTIDAD**: consumibles/cable/conectores. **SERIAL**: cámaras, NVR, switches, AP... (cada unidad rastreada por serie).
- El migrador consolida duplicados históricos y reapunta detalle de guías, salidas, movimientos, series y seguimiento al producto base.

11.5 · Normalización de estados

Regla automática: **cantidad > 0** → **En Stock**, **cantidad ≤ 0** → **Sin Stock**, **sin tocar** estados especiales (*Baja, Instalado, En Revisión, En Tránsito*). Se aplica al guardar cambios de stock y en el arranque para registros históricos.

12 · Tipos de movimiento (kardex)

Todo cambio de stock deja un registro en `movimientos` con stock antes/después y usuario.

Tipo	Cuándo se genera	Efecto en stock
INGRESO	Alta o incremento de stock (por cantidad o unidad serializada)	Aumenta
SALIDA_GUIA	Al emitir una guía de salida	Disminuye
SALIDA_DIRECTA	Salida directa sin guía	Disminuye
DEVOLUCION_GUIA	Al quitar series / devolver parcial de una guía	Aumenta
ANULACION_GUIA	Al anular una guía completa	Aumenta (reintegra)
BAJA	Al dar de baja un producto	Deja en 0
AJUSTE	Corrección manual de inventario	Variable
TRANSFERENCIA	Movimiento entre ubicaciones	Neutro

Cada registro guarda además la **referencia** (código de guía `GS-...` , `BAJA-...` , `ING-SN-...`), el **usuario** autenticado y una **observación** contextual. El kardex es la fuente de verdad para auditar quién movió qué y cuándo.

Trazabilidad end-to-end. Para un equipo serializado se puede reconstruir toda su historia: ingreso (unidad en `equipo_series`) → salida por guía (`guia_detalle_series` + movimiento `SALIDA_GUIA`) → devolución/anulación (vuelta a `EN_STOCK`) → baja. Cada paso queda fechado y con responsable.

13 · Capa de presentación

13.1 · Plantillas y JavaScript

19 vistas Jinja2 heredan de `base.html` (barra lateral, tema naranja corporativo, meta con el token CSRF). El comportamiento vive en dos archivos:

Archivo	Responsabilidad
<code>static/js/app.js</code>	Sidebar colapsable con preferencia persistida en <code>localStorage</code> (auto-colapso en pantallas $\leq 768\text{px}$ si no hay preferencia). Inyección CSRF : añade el token oculto a todos los formularios <code>POST</code> . Estado de carga : al enviar, deshabilita los botones y muestra un spinner, evitando el doble clic que generaba guías duplicadas.
<code>static/js/dashboard.js</code>	Filtro en vivo del inventario (marca, modelo, categoría, estado, SKU, serie, MAC) y paginación de 25 registros sobre el resultado filtrado.

13.2 · APIs de filtrado en cascada

Los formularios de ingresos y guías no usan datos estáticos: consultan la BD por API para filtrar dependientes.

Endpoint	Devuelve
<code>/api/catalogos</code>	Categorías (y datos base de catálogo).
<code>/api/marcas?categoria=</code>	Marcas asociadas a la categoría.
<code>/api/modelos?categoria=&marca=</code>	Modelos de esa cat+marca.
<code>/api/productos?...</code>	Productos con stock disponible según filtros.
<code>/api/series?equipo_id=</code>	Series EN_STOCK del producto (para guías serializadas).

13.3 · Generación de PDF (guía de salida)

`pdf_render.py` construye la guía con **ReportLab** (A4, Platypus) y devuelve un `BytesIO` que la ruta `/pdf_guia/<id>` envía con `send_file`. Incluye:

- Un **membrete** propio (flowable) con banda oscura, logo y el naranja exacto de la marca (`#E33F10`).
- Datos de cabecera (código `GS-000001`, estado, destino, solicitante, aprobaciones, fecha).
- Tabla de ítems con cantidades y, para productos serializados, el **listado de series** despachadas.
- Paleta y tipografía consistentes con la identidad "Portero Seguro".

La lógica de datos (queries de guía, detalle y series) vive en la ruta; el módulo de PDF solo maqueta. Así el render es testeable de forma aislada y no acopla ReportLab al resto de `app.py`.

14 · Mapa de rutas (endpoints)

Alrededor de 50 rutas. Rol mínimo entre paréntesis (G=GET, P=POST).

Autenticación y usuarios

/login	público
/logout (P)	sesión
/usuarios	admin*
/usuarios/crear	admin
/usuarios/<id>/editar	admin
/usuarios/<id>/resetear_password	admin

*el cambio de contraseña propio se hace en esta pantalla y lo puede usar cualquier usuario logueado.

Inventario e ingresos

/ (dashboard)	lectura
/ingresos	G lectura / P operador
/ingreso_series	G lectura / P operador
/editar_equipo/<id>	operador
/dar_baja_equipo/<id>	operador
/series · /series/<id>/eliminar	lectura / operador

APIs JSON

/api/catalogos marcas modelos	lectura
/api/productos series	lectura

Guías y salidas

/guías · /listar_guías	lectura
/guardar_guia	operador
/actualizar_guia/<id>	operador
/guia/<id>	lectura
/guia/<id>/series	G lectura / P operador
/guia/<id>/quitar_serie/<sid>	operador
/eliminar_guia/<id>	operador
/editar_guia/<id> · /pdf_guia/<id>	lectura
/salidas	G lectura / P operador

Operación en campo

/movimientos	lectura
/seguimiento (+ act./elim.)	operador
/avances (+ act./elim.)	operador

Administración

/configuracion	admin
/agregar_categoria marca modelo cargo	admin
/editar_catalogo · /eliminar_catalogo	admin
/personal (+ editar/eliminar)	G lectura / admin
/edificios (+ editar/eliminar)	G lectura / admin

15 · Despliegue y acceso en red

Un doble clic en `lanzar.bat` levanta todo el stack.



Formas de acceso

Forma	URL	Requiere
Por IP (siempre funciona)	<code>http://192.168.18.137</code>	Estar en la red local
Por dominio local	<code>http://inventario.porteroseguro.com</code>	Entrada DNS en el router (una vez)

Modelo actual: HTTP sin cifrado, elegido para no instalar certificados en cada equipo. Válido para **red local de confianza**. Para candado HTTPS sin configurar cada PC se necesitaría un dominio real (certificado Let's Encrypt).

Componentes de despliegue

Archivo	Rol
<code>lanzar.bat</code>	Orquesta: eleva a admin, verifica Python, instala dependencias, descarga Caddy, abre el firewall (puerto 80), registra el dominio en <code>hosts</code> y lanza el launcher.
<code>_portero_launcher.py</code>	Supervisa Caddy + Flask, captura las credenciales del primer arranque y muestra el banner con la URL e IP. Ctrl+C detiene ambos.
<code>Caddyfile</code>	Proxy inverso <code>:80 -> 127.0.0.1:5051</code> con gzip; responde por nombre o por IP.
<code>configurar_cliente.bat</code>	<i>Fallback</i> : registra el dominio en el <code>hosts</code> de un cliente si el router no permite DNS local.

Variables de entorno

Variable	Defecto	Descripción
<code>SECRET_KEY</code>	generada	Firma de sesiones (fijarla en producción).
<code>ADMIN_USERNAME</code> / <code>ADMIN_PASSWORD</code>	admin / aleatoria	Credenciales del admin inicial.
<code>FLASK_HOST</code> / <code>FLASK_PORT</code>	127.0.0.1 / 5051	Interfaz y puerto de Flask.
<code>FLASK_DEBUG</code>	false	Modo debug (nunca en producción).
<code>BEHIND_PROXY</code>	true (launcher)	Activa ProxyFix (IP real del cliente).
<code>SESSION_COOKIE_SECURE</code>	false (HTTP)	Cookies solo por HTTPS.
<code>PORTERO_DB</code>	inventario.db	Ruta de la BD (bases separadas para tests).
<code>XDG_DATA_HOME</code>	caddy_data	Directorio de datos de Caddy.

16 · Estructura, pruebas, roadmap y apéndices

Estructura del proyecto

```
proyecto_mejorado/
├─ app.py                # Monolito: rutas + negocio (~3.450 líneas)
├─ db.py                 # Datos: conexión, esquema, migraciones, catálogos
├─ auth.py               # Seguridad: hashing, roles, CSRF
├─ pdf_render.py         # PDF de guías (ReportLab)
├─ migrar_fase1.py       # Consolidaciones históricas
├─ requirements.txt      # Flask, reportlab
├─ lanzar.bat            # Arranque orquestado (Windows)
├─ _portero_launcher.py  # Supervisor de Caddy + Flask
├─ Caddyfile             # Config del proxy inverso
├─ configurar_cliente.bat # Fallback DNS para clientes
├─ inventario.db         # Base SQLite (no versionada)
├─ templates/ (19 vistas Jinja2)  static/ (js · img)
├─ docs/ (modelo ER, manuales, este PDF)
├─ tests/ (test_flujos.py)      logs/ (app.log, caddy.log)
```

Pruebas automatizadas

Suite de integración `tests/test_flujos.py` (10 pruebas): `python -m unittest tests.test_flujos -v`.
Cubren: redirección de rutas protegidas sin sesión, login correcto/incorrecto, rechazo de `POST` sin CSRF, bloqueo por rate limiting, restricción del rol `lectura` y 4 pruebas del descuento atómico de stock (incluida la concurrencia). Usan una BD temporal vía `PORTERO_DB`: **nunca tocan `inventario.db`**.

Roadmap

- Separar las rutas en **blueprints** (`inventario`, `guias`, `admin`).
- Migrar categoría/marca/modelo de texto libre a **claves foráneas numéricas**.
- Sistema de **migraciones numeradas** en vez de `init_db` en cada arranque.
- Expiración/renovación de sesión por inactividad.
- **HTTPS con candado** mediante dominio real + certificado Let's Encrypt.

Apéndice A · Estados

Entidad	Estados
Equipo	En Stock · Sin Stock · Baja · Instalado · En Revisión · En Tránsito
Serie	EN_STOCK · ENTREGADO · BAJA
Guía	ACTIVA · ANULADA
Seguimiento	EN_SEGUIMIENTO · RETIRADO · INSTALADO · DEVUELTO · BAJA
Avance	PENDIENTE · EN_PROCESO · TERMINADO · OBSERVADO · CANCELADO

Apéndice B · Códigos de referencia

Prefijo	Significado
<code>GS-000001</code>	Guía de salida
<code>BAJA-000001</code>	Baja de producto
<code>ING-SN-000001</code>	Ingreso de unidad serializada

Apéndice C · Comandos útiles

```
python -m unittest tests.test_flujos -v
copy inventario.db backup.db # respaldo
lanzar.bat                    # arrancar todo
```